kotlin-particles: Top 10 Improvements

Scope: Engine / CommonEngine, WasmJsEngine, WasmJsGpuEngine, AndroidEngine, ParticleFactory / CommonParticleFactory, SpatialGrid / CommonSpatialGrid.

Headline: drawGroups, drawBubbles and drawProjectileBlobs disappear once every particle carries a RenderKind (SIMPLE_SHAPE, COMPLEX_SHAPE, SPRITE, ANIMATED_SPRITE). Bubble and blob are both complex shapes. drawParticles then has no knowledge of effect types.

1. Dispatch drawing by RenderKind: SIMPLE_SHAPE, COMPLEX_SHAPE, SPRITE, ANIMATED_SPRITE (removes drawGroups / drawBubbles / drawProjectileBlobs)

WasmJsEngine.draw() and drawParticles() branch on effect (BUBBLE, ITEM_RETURN, ANIMATED_ITEM_RETURN, BLOB_PROJECTILE) and pre-sort particles into lists; AndroidEngine duplicates it. Give every particle one of four render kinds and one renderer per kind:

```
enum class RenderKind { SIMPLE_SHAPE, COMPLEX_SHAPE, SPRITE, ANIMATED_SPRITE }

interface KindRenderer {
    fun submit(particle: Particle)   // batch or draw
    fun flush(canvas, viewPort)
}

SimpleShapeRenderer   circle/rect, batched per colour into one path (today's drawGroups)
ComplexShapeRenderer  per-particle draw, delegates to particle.complexShape (bubble, blob)
SpriteRenderer        drawImageRect, frame fixed at 0 (ITEM_RETURN)
AnimatedSpriteRenderer same, frame = particle.frame (ANIMATED_ITEM_RETURN)
```

drawParticles becomes: cull, renderers[particle.renderKind].submit(p), then flush in fixed z-order. No effect checks and no helper lists. SPRITE and ANIMATED_SPRITE can share one implementation (frame 0 vs particle.frame) if you want only three code paths.

2. Treat bubble and blob as ComplexShape implementations, not separate engine paths

A bubble is a complex shape (gradient fill, rim stroke, highlight) and so is the projectile blob (lobed spline path). Define a small strategy that ComplexShapeRenderer calls:

```
interface ComplexShape {
    fun draw(surface: DrawingSurface, particle: Particle, offsetX: Float, offsetY: Float)
}
object BubbleShape : ComplexShape { ... }   // was drawBubbles
object BlobShape   : ComplexShape { ... }   // was drawProjectileBlobs + buildSmoothClosedBlobPath
```

The factory sets renderKind and complexShape when creating the particle (Particle gets one nullable/enum field, e.g. complexShape: ComplexShapeType). Adding a new complex look is one new class. The PROJECTILE_BLOB_* constants move out of CommonParticleFactory into BlobShape, since they are render concerns.

3. Make the platform split a thin canvas adapter (dedupe WasmJsEngine / AndroidEngine)

Both engines carry identical logic (culling, alpha*255 colour packing, blob lobe math, bubble gradient math, spline builder) and differ only in Skia vs android.graphics calls. Define a small DrawingSurface interface (drawOval, drawRect, drawPath, drawCircle, linearGradient, radialGradient, drawImageRect) and move all geometry and the renderers from item 1-2 to commonMain. Only the adapter is platform code.

4. Eliminate per-frame allocations in the draw path

drawParticles allocates a HashMap, two ArrayLists, a PathBuilder per colour and a Path per group every frame; bubbles create Shader objects per particle; blobs allocate two FloatArrays and a Path per particle; draw() builds bubble/remaining lists each frame. Reuse cleared collections owned by each renderer, cache gradient shaders

(unit-radius shader plus canvas translate/scale), reuse one Path (reset()) and the lobe arrays, and take List instead of ArrayList. SIMPLE_SHAPE batching could sort in place by colour and flush runs instead of using a map.

5. Break CommonEngine's 'throw must be implemented' pattern into real abstractions

initialize(), manageParticles() and draw() are open methods that throw, deferring errors to runtime; make them abstract or provide the shared logic. The abstract vars typed Any (foregroundSurface, mapElementPaint, particlePaint, mapItemReturnPaint, getOrCreateForegroundSurface) force casts like 'particlePaint as Paint' in every subclass. With renderers owning their paints, drop them from the base class or make it generic. mapItemFrameWidth/Height move into the sprite renderers. Expose completedParticleResults as a read-only view.

6. Split CommonParticleFactory (1350 lines) into per-effect spawners

One class holds ~140 constants, 12 create* functions, GPU buffer packing and colour utilities. Introduce interface EffectSpawner { fun spawn(request: SpawnRequest): MutableList } with one class per effect (DustSpawner, BubbleSpawner, BlackHoleSpawner...) owning its constants, plus a Map registry. create() becomes a lookup instead of a 13-branch when, and the repeated 'destinationX ?: rightEdge' defaults collapse into SpawnRequest. Each spawner declares its RenderKind.

7. Move GPU packing out of the ParticleFactory interface (interface segregation)

ParticleFactory exposes createGpuParticleComputeBuffer and writeGpuParticleSpawnBuffer with default bodies that throw EngineException, so CPU engines inherit methods they cannot use. Extract GpuParticleBufferWriter, injected only into GPU engines. The magic effect type ids (1f..13f, 24 floats per particle) should come from one shared descriptor that the writer and shader reference, replacing the chained 'particle.effect == X ->' mapping.

8. Put per-effect traits in one descriptor instead of if/else on effect in engines

Draw, spawn, GPU type mapping and the selectedParticleEffect == BUBBLE check in draw() each know about effects. One descriptor per effect (renderKind, complexShape, gpuTypeId, spawner, usesSpatialGrid, drawOrder) keeps engines effect-agnostic. Once rendering is kind-based, selectedParticleEffect can be removed from the Engine interface.

9. SpatialGrid: remove allocations, lambda overhead and tuple destructuring in the hot path

forEachNearbyPair invokes a (Particle, Particle) -> Unit lambda O(n*k) times and iterates FORWARD_NEIGHBOR_OFFSETS as Pair (boxed, destructured per cell). Use two IntArrays for offsets, make the method inline or index-based, and use indexed loops (for-in over MutableList allocates iterators). Replace Array> with a flat counting-sort layout (cellStart IntArray + particle index IntArray) so beginTick does not clear thousands of lists. Also, cellIndexFor returns -1 for off-viewport particles, silently dropping them from collision; consider a padded margin.

10. SpatialGrid: sizing feedback loop and API clean-up

Cell size comes from the previous tick's max reach, so one large attractor (BLACK_HOLE / GOBBLER, attraction 480*mass) inflates cells for every particle, and any size change reallocates the whole cell array. Add hysteresis (resize only on >25% change), cap the cell size, handle long-reach particles in a separate list, and expose cell size/columns/rows/origin as an immutable snapshot. The grid is mutable state in an @AppScope singleton and not thread-safe; consider per-simulation instances, and add tests for pair uniqueness across cell borders (forward offsets 1,0 / -1,1 / 0,1 / 1,1).